

SceneFlow

AI Storyboard Intelligence - Infosys Specialist Programmer Interview Prep

30-second project pitch

SceneFlow turns a full script into a visual storyboard pipeline. A user submits a script, Gemini segments it into coherent scenes, each scene is analyzed into visual search intent, images are fetched concurrently from Pexels, Pixabay, and Openverse, semantic retrieval happens through Qdrant, a deterministic reranker selects relevant assets, and Celery + Redis keeps the long-running work off the FastAPI request thread. The final ranked results are persisted so previous searches can be reopened without silently rerunning the pipeline.

Technology Stack at a Glance

Layer	Technology	Why it was used
Frontend	React + TypeScript + Vite + Tailwind	Fast iteration, typed UI, responsive workspace
API	FastAPI + Pydantic	Typed APIs, validation, async-friendly request layer
Primary DB	PostgreSQL + SQLAlchemy + Alembic	Relational integrity, migrations, persistent history
Background jobs	Celery + Redis	Decouple long-running search/AI work from HTTP requests
Vector search	Qdrant + Sentence Transformers	Semantic retrieval over normalized asset metadata
AI	Gemini	Structured script segmentation + scene intelligence
Asset sources	Pexels + Pixabay + Openverse	Multi-provider visual retrieval with failure isolation

1. What problem does your project solve?

Answer

SceneFlow reduces the manual work of turning a written script into a visual storyboard. Instead of manually breaking the script into scenes, writing image searches, checking multiple stock sites, and choosing assets one by one, the system creates scene-level visual intent and retrieves and ranks candidate assets automatically.

2. Why did you build it?

Answer

I wanted to build a project where AI is part of a real application pipeline, not just a chatbot feature. The project combines structured LLM output, relational persistence, background processing, semantic vector search, multi-provider APIs, deterministic ranking, and a usable frontend. It gave me a practical way to study how these components work together.

3. What exactly did you implement?

Core implemented capabilities:

- Script segmentation: Gemini converts a full script into ordered, granular scene records with titles and preserved source text.
- Scene intelligence: Gemini extracts summary, subjects, actions, environment, mood, time context, and visual-search queries.
- Multi-provider retrieval: Pexels, Pixabay, and Openverse are normalized behind one provider interface and queried concurrently.
- Semantic retrieval: Sentence Transformers create embeddings and Qdrant performs vector search with metadata filters.
- Deterministic reranking: semantic relevance, resolution, and orientation are combined into an explainable final score with stable tie-breakers.
- Background processing: Celery + Redis moves long-running searches out of the synchronous HTTP request cycle.
- Historical results: ranking scores and search context are persisted so completed jobs can be reopened without a fresh Qdrant/Gemini search.
- Frontend workspace: React/TypeScript UI for projects, scripts, scenes, AI analysis, job polling, search history, and visual results.

4. Why these technologies?

FastAPI

I needed a typed API layer with clean dependency injection and good support for async I/O. It also works well with Pydantic models for strict API contracts.

PostgreSQL

The domain is relational: User -> Project -> Script -> Scene -> SearchJob -> Asset. PostgreSQL gives strong constraints, foreign keys, indexes, transactions, and migrations.

Celery + Redis

Provider APIs, Gemini, embeddings, and Qdrant can make search slow or failure-prone. Celery separates that work from the HTTP request, while Redis acts as the broker.

Qdrant

Keyword matching is not enough for visual intent. Qdrant lets us retrieve semantically similar assets using embeddings and metadata filters such as orientation.

Sentence Transformers

For this local prototype, a compact embedding model gave us a simple, low-cost way to generate vectors locally without another paid embedding API.

Gemini

Structured output is useful for both script segmentation and scene analysis because downstream services need predictable fields rather than free-form prose.

React + TypeScript

The product is interaction-heavy: scene navigation, job polling, search history, and image cards. TypeScript helps keep frontend/API contracts aligned.

Alembic

Database changes need reproducible migrations rather than manual schema edits.

5. What was the hardest part?

Answer

The hardest part was making the pipeline reliable across multiple asynchronous and stateful systems. The flow crosses Gemini, three external providers, embeddings, Qdrant, PostgreSQL, Celery, and Redis. The challenge was not just making each component work independently; it was making failures, retries, transactions, ranking, and historical results behave consistently.

6. What bug took the most time?

Answer

One of the most important bugs was ranking scores showing as 0.000 even though the ranking service had calculated real values. The root cause was that Asset ORM objects were no longer attached to the session that performed the final commit. The in-memory objects changed, but SQLAlchemy did not persist those mutations. I fixed it by re-querying the Asset rows in the active session before updating and committing the scores. That bug reinforced an important lesson: changing an ORM object in Python does not guarantee the database will persist the change unless the object is tracked by the active session and the transaction is committed.

7. How did you debug it?

Debugging method:

- Reproduced the issue through a real end-to-end search instead of relying only on unit tests.
- Inspected the API response and saw `final_score/feature` values were zero.
- Queried the PostgreSQL Asset rows directly to determine whether the problem was in ranking, persistence, or serialization.
- Traced the Celery task from candidate creation through `RankingService` and database commit.
- Compared the SQLAlchemy object/session lifecycle and identified detached ORM instances.
- Changed the persistence step to re-load the Asset rows in the active session, update the score fields, and commit.
- Ran the full regression suite and then verified a brand-new job with real persisted values.

8. How does the request flow through the system?

Search request flow

React -> POST /scenes/{scene_id}/search -> FastAPI creates a SearchJob and returns 202 Accepted
-> Celery task is enqueued through Redis -> Gemini generates scene/search intelligence -> Pexels + Pixabay + Openverse are called concurrently -> assets are normalized and persisted -> embeddings are generated -> Qdrant retrieves semantic candidates -> RankingService reranks them -> ranking scores are persisted in PostgreSQL -> SearchJob becomes COMPLETED -> React polls the job status -> Results page reads the historical persisted results from PostgreSQL.

Key idea: the HTTP request is fast; expensive work happens asynchronously.

Stage	Responsibility	Main component
1	Accept request + create job	FastAPI + PostgreSQL
2	Queue background work	Celery + Redis
3	Understand visual intent	Gemini
4	Fetch candidate assets	Pexels / Pixabay / Openverse
5	Semantic retrieval	Sentence Transformers + Qdrant
6	Business reranking	RankingService
7	Persist historical result	PostgreSQL
8	Poll + render	React

9. How is data stored?

The core relational hierarchy is:

User	Project	Script	Scene	SearchJob	Asset
------	---------	--------	-------	-----------	-------

- PostgreSQL: authoritative relational storage for domain data.
- UUID primary keys: non-sequential identifiers for API-facing resources.
- Foreign keys + cascades: preserve referential integrity across the hierarchy.
- Indexes: created on foreign keys and frequently queried fields such as job status and provider metadata.
- SearchJob: stores requested_query, ranking_version, status, error_message, and execution timestamps.
- Asset: stores provider metadata plus persisted semantic/resolution/orientation/final scores so historical result pages are stable.
- Qdrant: stores vector representations and retrieval metadata used for semantic candidate retrieval; PostgreSQL remains the authoritative domain store.

10. How is authentication handled?

Current implementation - be honest in the interview

Real JWT authentication is not implemented yet. The current V2 prototype isolates identity behind a FastAPI dependency that resolves a development user from configuration. This lets the rest of the application be built against a consistent user identity boundary. The intended production change is to replace that dependency with JWT/OAuth2 token validation without rewriting the CRUD routes.

Do not say: "Authentication is fully implemented."

Say instead: "Authentication is the remaining security phase; currently the architecture has a dependency boundary for current-user resolution, and production would replace it with JWT validation and authorization checks."

11. What would you change for production?

- Authentication/authorization: implement JWT/OAuth2, ownership checks, refresh/revocation strategy, and rate limits.
- Infrastructure: containerize the full stack and deploy FastAPI/Celery/Redis/Qdrant/PostgreSQL with separate production configurations.
- Observability: structured logging, distributed tracing, metrics, provider latency tracking, and error monitoring.
- Reliability: idempotency keys, stronger retry policies, dead-letter handling, timeouts/circuit breakers, and worker concurrency controls.
- Data/storage: store durable asset snapshots or object-storage references if external CDN URLs are not reliable enough.
- Security: secret management, stricter CORS, API-level authorization, input limits, dependency scanning, and automated security checks.

- Testing: dedicated PostgreSQL test infrastructure, contract tests for external providers, and full CI/CD.
- AI operations: prompt/version evaluation, model configuration management, quota handling, caching, and offline regression datasets.
- Search quality: evaluate ranking with labeled examples instead of relying only on hand-tuned heuristic weights.

12. Architecture decisions worth explaining

Decision	Reason
Separate GeminiScriptSegmenter and GeminiSceneAnalyzer	Different responsibilities: Script -> Scenes vs Scene -> Visual Intelligence.
External prompt Markdown files	Prompt engineering can evolve without mixing long instructions with Python business logic; Git versions the prompts.
Provider abstraction	Pexels/Pixabay/Openverse differences stay inside provider adapters; the rest of the pipeline sees one normalized schema.
Two-stage retrieval + reranking	Qdrant provides fast semantic recall; deterministic ranking applies product-specific business preferences after retrieval.
Persist ranking scores	Historical results must not silently change when ranking logic or Qdrant contents change.
Celery background jobs	Long-running AI/search work should not block HTTP requests.
PostgreSQL as source of truth	Domain data and historical result snapshots belong in a transactional relational store; Qdrant is a retrieval index.

13. Common follow-up questions

Why not use only one image provider?

Multiple providers improve coverage and resilience. Provider failures are isolated so one timeout does not kill the whole search.

Why Qdrant instead of SQL LIKE queries?

Semantic similarity captures meaning, while keyword matching can miss related phrasing. SQL is still used for authoritative metadata and hydration.

Why not store vectors in PostgreSQL?

Qdrant is specialized for vector retrieval. PostgreSQL remains the system of record; Qdrant acts as the semantic index.

Why use heuristics after vector search?

Semantic similarity optimizes relevance, but product preferences such as orientation and resolution are not always represented well by embedding similarity alone.

Why persist ranking scores?

Otherwise reopening an old search could produce different results after model/index/ranking changes. Persistence turns a completed job into a historical snapshot.

What happens if one provider fails?

The aggregator isolates provider errors. Successful providers still contribute results; the job only fails when the overall operation cannot produce a viable result set.

Why 202 instead of 200 for search?

The work is accepted but not complete. 202 communicates asynchronous processing and lets the client poll job state.

What happens if Celery receives the same job twice?

The worker checks the SearchJob state and uses an idempotency guard so completed/running jobs are not processed again.

14. Interview-ready closing statement

60-second closing

"The interesting part of SceneFlow is the complete pipeline. I did not stop at calling Gemini. I built a system where a script is segmented into scenes, each scene gets structured visual intent, multiple image providers are queried concurrently, semantic retrieval happens through Qdrant, deterministic reranking explains why an asset is selected, and Celery keeps the long-running work asynchronous. I also persisted the ranking result so historical searches remain stable. The main remaining production step is real authentication and the operational hardening around deployment, observability, and reliability."

Important honesty checkpoint

The current project has not completed real JWT authentication. Also, at the time of the latest segmentation phase, live Gemini smoke testing was blocked by the free-tier quota; mocked tests were used for the segmentation code paths. Do not claim either was fully production-verified if asked.